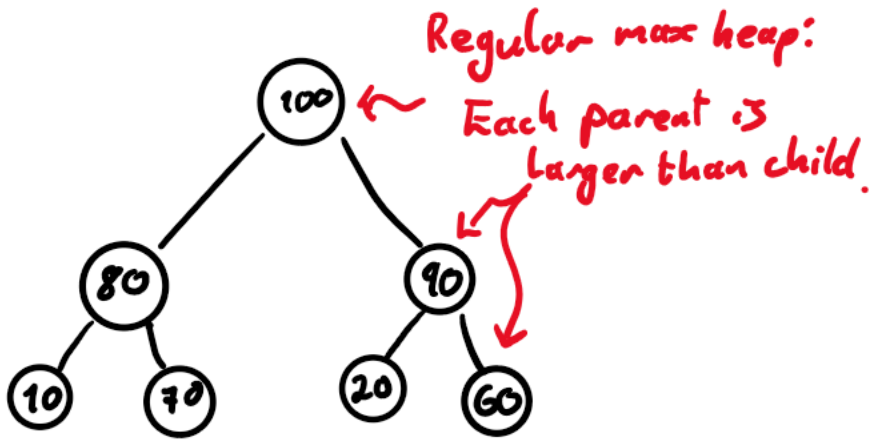
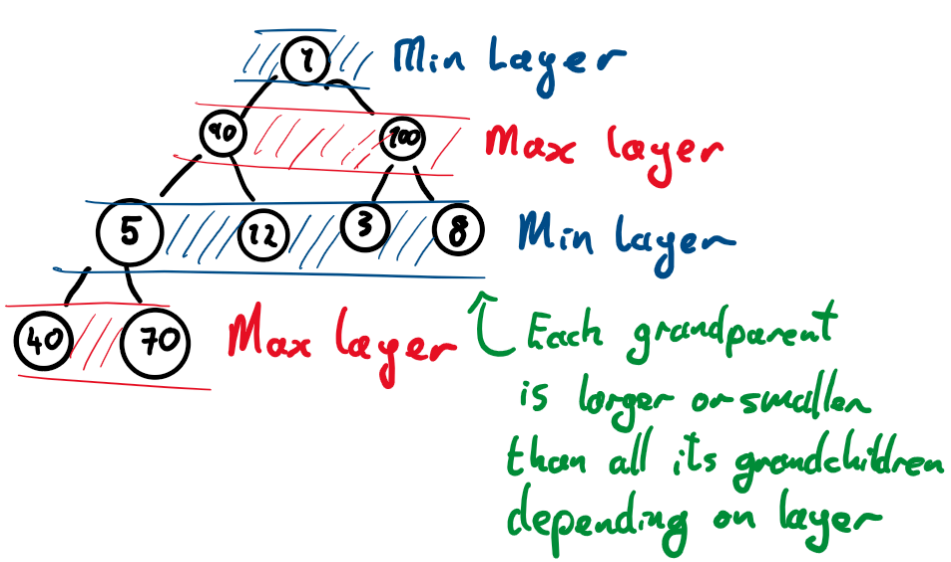
Double-Ended Priority Queue

Most priority-queue implementations rely on organising keys into a binary heap and following a specific algorithm when inserting keys such that the top of the heap is always known to be the highest priority key in the data structure. This allows O(1) fetching the highest priority key but O(n) for keys of the opposite priority. This O(n) fetching of the other priority struck me as odd so I took it upon myself to design a data structure that could do both in O(1). The fastest way of doing this is storing everything twice into a high priority queue and a low priority queue. To avoid this wasted storage I came up with a binary heap where it's split into alternating layers: each min-level node is the smallest value in its subtree, and each max-level node is the largest in its subtree.



Regular binary heap



Double-layered heap

In our specific example on the right, the minimum key is always on the top whilst the maximum key is one of its two immediate children. This allows for O(1) fetching of both sides of the priority queue. The actual logic of how to make insertions and deletions becomes problematic because you have to reshuffle based on grand-descendants rather than immediate children. The implementation of the data structure is given here (please note that it's possibly broken as I attempted to introduce C++20 & C++23 features after finishing without saving the original complete data structure. The logic is correct, just syntactically problematic)

https://github.com/Adam76e54/DataStructures/blob/main/MinMaxHeap/MinMaxHeap.h